Parameter-Efficient Iterative Transformers for Compositional Binary Search Tree Code Generation

George Yuan
1012152224, zhijie.yuan@mail.utoronto.ca
GitHub: https://github.com/ShiinaMahiruOvO/aps360-bst-code-lm

Introduction

Small language models can produce familiar code patterns, yet often fail when a prompt combines several individually simple requirements. We study this problem in a controlled setting: generating correct Python implementations of binary-search-tree (BST) operations from natural-language specifications. Our central question is whether repeatedly applying a shared Transformer core, while allowing low-rank parameters to specialize each iteration, improves *compositional generalization* without substantially increasing the parameter count.

We will pretrain an approximately 100M-parameter decoder-only model on educational Python code, then fine-tune it on automatically generated and unit-tested BST tasks. We will compare a standard Transformer, a looped Transformer, and a looped Transformer with iteration-specific LoRA adapters. Deep learning is appropriate because the input and output are variable-length sequences and the desired mapping must capture both natural-language constraints and long-range program dependencies; fixed templates or rule-based systems would not cover the variation in valid programs. Success is measured by functional correctness: a completion passes only if it satisfies every hidden test. We will report results separately for atomic tasks, familiar combinations, and held-out combinations, rather than relying only on token-level loss.

This is an individual project. I will implement and document the data pipeline, model architecture, training procedures, execution-based evaluation, and final analysis. Configuration files, fixed random seeds, and version-controlled scripts will be used to keep the experiments reproducible.

Illustration

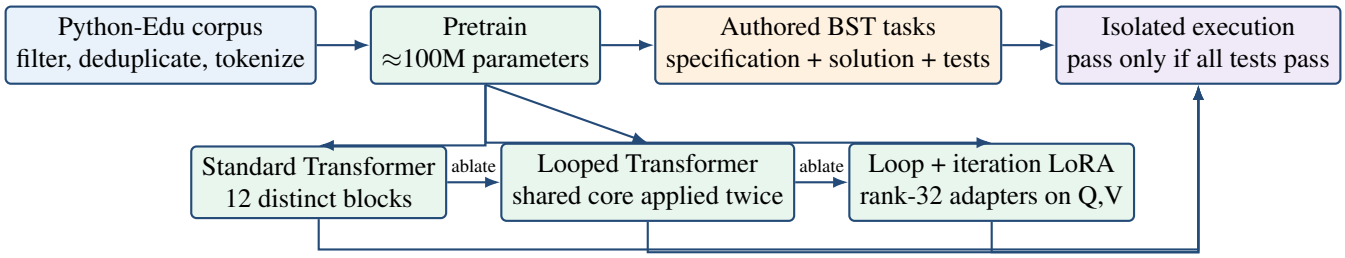


Figure 1: Project pipeline. The primary experiment changes the model's depth-sharing mechanism while holding the tokenizer, data, and evaluation protocol fixed.

Background & Related Work

The Transformer established attention-only sequence modelling and is our standard baseline [7]. Universal Transformers introduced recurrent application of a shared block, motivating our looped core and suggesting potential benefits on algorithmic tasks [3]. RoPE injects relative position information by rotating query and key features and is used in all variants [6]. LoRA showed that trainable low-rank updates can specialize a frozen or shared weight matrix with few additional parameters [4]; we instead use separate low-rank updates to distinguish successive applications of the same core. Codex and HumanEval established execution-based functional correctness as a more meaningful code-generation metric than textual similarity [2]. Finally, GRPO replaces a learned value model with within-group reward normalization [5]. We treat execution-reward training as a secondary extension after establishing the supervised architecture comparison.

Data Processing

For pretraining, we use the Python-Edu subset of the public SmolLM-Corpus [1], which contains Python files selected for educational quality. Our pipeline removes empty or extremely short files, generated/minified content, malformed or non-Python records, and exact duplicates; it also rejects samples that fail Python syntax parsing. Repositories, rather than individual files, are assigned to train/validation splits to reduce leakage. We train an 8,192-token byte-pair tokenizer and serialize token IDs into memory-mapped shards. The current processed corpus contains approximately 1.03M training documents (498M tokens) and 55M validation tokens. Scripts and configuration files record every filter, seed, split, and tokenizer setting.

Domain fine-tuning data is generated by us rather than copied directly from a benchmark. A task generator combines 17 BST operation contracts—including search, insertion, predecessor/successor, range count, range sum, and path queries—with explicit edge-case and return-format conditions. Each record contains a prompt, a reference implementation, and deterministic unit tests. We currently have 552 training examples, 87 validation examples, and an 87-task evaluation set (56 atomic/basic and 31 multi-condition tasks). A separate 27-task compositional holdout contains unseen three- or four-condition combinations. We prevent exact prompt and solution overlap, split by specification before rendering templates, and retain the holdout only for final evaluation. Reference solutions must pass all generated tests inside a time-limited subprocess.

Architecture

All models are decoder-only Transformers with an 8,192-token vocabulary, context length 1,024, hidden width 768, 12 attention heads, RMSNorm, RoPE, and SwiGLU. The standard baseline has 12 distinct blocks ($\approx$ 97.5M parameters). The looped model uses two prelude blocks, an eight-block shared core executed twice, and two coda blocks: it has the same 12 distinct blocks but an effective depth of 20. Our final model adds rank-32 iteration-specific LoRA updates to the query and value projections in the shared core ($\approx$ 99.1M parameters). This tests whether the two passes can learn different transformations while retaining most parameter sharing. All variants use the same pretraining and answer-only supervised fine-tuning pipeline. If time permits, we will apply execution-reward GRPO only to the best supervised model.

Baseline Model

The primary baseline is the standard 12-layer Transformer trained with the same corpus, tokenizer, optimizer family, fine-tuning examples, and decoding budget. This gives a reproducible, approximately parameter-matched comparison; the looped model and loop+LoRA model form two controlled ablations. We will report total and subset pass rates, parameter count, and training/inference cost. The primary success criterion is at least a 10 percentage-point improvement over the standard model on multi-condition tasks without a drop greater than five points on atomic/basic tasks. The untouched 27-task holdout is reported separately to test whether gains transfer to unseen combinations. A public 0.5B code model may be included as a scale-mismatched sanity check, but it is not the main baseline.

Ethical Considerations

Training on public code creates licensing, attribution, privacy, and memorization risks; dataset provenance is therefore documented, duplicates are removed, and model outputs will not be presented as original human work. Generated programs can be subtly incorrect or insecure, so our model is an educational prototype rather than a deployment-ready programming assistant. Untrusted completions are executed only in resource-limited subprocesses with timeouts and no network access. We will report failures and possible benchmark contamination, avoid claims beyond the narrow BST domain, and release model/data documentation that states these limitations.

References

[1] Loubna Ben Allal, Anton Lozhkov, Elie Bakouch, Leandro von Werra, and Thomas Wolf. Smollm-corpus. Hugging Face dataset, 2024. URL https://huggingface.co/datasets/HuggingFaceTB/smollm-corpus.

[2] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[3] Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Lukasz Kaiser. Universal transformers. In *International Conference on Learning Representations*, 2019.

[4] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

[5] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

[6] Jianlin Su, Yu Lu, Shengfeng Pan, Bo Wen Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021.

[7] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.